Primer: Nested Fixed Point (NFXP-NK)

Rust (1987), Iskhakov, Rust and Schjerning (2016)

econirl package documentation

1 Context and Problem

Harold Zurcher manages a fleet of buses and must decide each period whether to keep operating the current engine or pay a fixed cost to replace it. Rust (1987) formalized this as a dynamic discrete choice (DDC) model and proposed the first computationally feasible estimator: the nested fixed point algorithm. The outer loop maximizes the log-likelihood over structural parameters θ. The inner loop solves the Bellman equation to machine precision at each candidate θ, producing the choice probabilities that enter the likelihood.

The inner loop is the bottleneck. Value iteration converges at rate β per step, so when β approaches 1 (patient agents, long horizons), convergence slows dramatically. At $\beta = 0.9999$, successive approximation requires thousands of contractions per likelihood evaluation.

Iskhakov, Rust and Schjerning (2016) introduced the SA-then-NK polyalgorithm that eliminates this bottleneck. The algorithm starts with successive approximation (SA) for safe global convergence and switches to Newton-Kantorovich (NK) near the fixed point for quadratic convergence. The total iteration count becomes insensitive to β: roughly 20 SA steps followed by 4 NK iterations suffices at any discount factor. In Monte Carlo experiments at $\beta = 0.9999$, NFXP-NK converges in 100 percent of 1,250 trials in 0.074 seconds, while MPEC-AMPL converges in zero.

NFXP is the foundational structural estimator in DDC. Every other estimator in this package defines itself relative to NFXP: CCP avoids the inner loop via Hotz-Miller inversion, MPEC replaces it with constrained optimization, NNES approximates it with a neural network, and TD-CCP eliminates the need for transition probabilities entirely.

2 Model and Notation

2.1 Notation

Symbol	Meaning
$s \in \mathcal{S}$, $a \in \mathcal{A}$	Discrete state and action
$\beta \in [0, 1)$	Discount factor
$\sigma > 0$	Logit scale parameter (Type I EV noise)
$P(s' \mid s, a)$	Transition kernel (known or estimated)
$\phi(s, a) \in \mathbb{R}^K$	Feature vector for state-action pair
$\theta \in \mathbb{R}^K$	Structural parameters
$u(s, a; \theta) = \theta^\top \phi(s, a)$	Flow utility (linear in parameters)
T	Soft Bellman operator (eq. 4)
$V^*(s; \theta)$	Integrated value function (fixed point of T)
$Q(s, a; \theta)$	Choice-specific value function
$\pi(a \mid s; \theta)$	Softmax choice probability
P_π	Policy-weighted transition matrix
B_k	BHHH Hessian approximation at step k

2.2 The DDC Model

An agent observes state s_t, chooses action a_t, and receives flow payoff $u(s_t, a_t; \theta) + \sigma\, \varepsilon_{t,a}$ where $\varepsilon_{t,a}$ is i.i.d. Type I extreme value. The choice-specific value function satisfies the Bellman equation

$$Q(s, a; \theta) \;=\; u(s, a; \theta) \;+\; \beta \sum_{s' \in \mathcal{S}} P(s' \mid s, a)\, V(s'), \tag{1}$$

where the integrated value function is the log-sum-exp

$$V(s) \;=\; \sigma \log \sum_{a \in \mathcal{A}} \exp\bigl(Q(s, a; \theta)/\sigma\bigr). \tag{2}$$

The optimal policy is the softmax

$$\pi(a \mid s; \theta) \;=\; \frac{\exp\bigl(Q(s, a; \theta)/\sigma\bigr)}{\sum_{a'} \exp\bigl(Q(s, a'; \theta)/\sigma\bigr)}. \tag{3}$$

Substituting (1) into (2) defines the soft Bellman operator T:

$$(TV)(s) \;=\; \sigma \log \sum_{a} \exp\Bigl(\bigl[u(s, a; \theta) + \beta \textstyle\sum_{s'} P(s'|s, a)\, V(s')\bigr]/\sigma\Bigr). \tag{4}$$

The operator T is a contraction with modulus β under the sup norm, so $V = TV$ has a unique fixed point for each θ.

2.3 Maximum Likelihood Objective

Given panel data $\mathcal{D} = \{(s_i, a_i)\}_{i=1}^N$ from I individuals observed over T periods ($N = I \times T$), NFXP estimates θ by maximizing the conditional log-likelihood

$$\hat{\theta}_{\text{NFXP}} \;=\; \arg\max_{\theta}\; \ell(\theta) \;=\; \arg\max_{\theta} \sum_{i=1}^{N} \log \pi(a_i \mid s_i; \theta). \tag{5}$$

The choice probabilities $\pi(a \mid s; \theta)$ depend on θ through the fixed point $V^*(\theta)$, which must be recomputed at each candidate θ during optimization.

3 Identification and Theory

Proposition 1 (MLE consistency)**.** *Under the assumptions of additive separability, conditional independence, and i.i.d. Type I extreme value errors (AS/CI/EV), if the feature matrix $\Phi \in \mathbb{R}^{|\mathcal{S}||\mathcal{A}|\times K}$ has full column rank and the parameter space Θ is compact with θ_0 in the interior, then $\hat{\theta}_{NFXP}$ is consistent and asymptotically efficient:*

$$\sqrt{N}\big(\hat{\theta} - \theta_0\big) \;\xrightarrow{d}\; \mathcal{N}\big(0,\; I(\theta_0)^{-1}\big), \tag{6}$$

where $I(\theta_0) = \mathbb{E}\big[\psi_i(\theta_0)\,\psi_i(\theta_0)^\top\big]$ is the Fisher information matrix and ψ_i is the per-observation score defined in (7).

The rank condition on Φ requires that features vary across actions. State-only features that are identical for all actions produce collinear columns and a singular information matrix. In the Rust bus example, the operating cost feature equals the mileage state for the keep action and zero for the replace action, while the replacement cost feature is one for replace and zero for keep. These features are action-dependent by construction.

The discount factor β is not identified without a priori restrictions. Rust (1987) fixes β and estimates θ conditional on β. The scale parameter σ is also not separately identified from the magnitude of θ; the convention is to normalize $\sigma = 1$.

For counterfactual analysis, the identified object $\hat{\theta}$ supports policy simulation under parameter perturbation. Changing θ (for example, increasing the replacement cost) while holding transitions P fixed produces a new policy $\pi'(a|s;\theta')$ by re-solving the Bellman equation. The parametric form $u = \theta^\top\phi$ resolves the reward shaping ambiguity: the reward is pinned down by θ and ϕ, so there is no free shaping potential.

4 Estimation Algorithm

The gradient of the log-likelihood is computed analytically via the implicit function theorem, differentiating through the Bellman fixed point without backpropagation.

The per-observation score is

$$\psi_i(\theta) \;=\; \frac{1}{\sigma}\Big[\nabla_\theta Q(s_i, a_i) - \sum_a \pi(a \mid s_i)\,\nabla_\theta Q(s_i, a)\Big], \tag{7}$$

where $\nabla_\theta Q(s,a) = \phi(s,a) + \beta\sum_{s'} P(s'|s,a)\,\frac{\partial V(s')}{\partial\theta}$ and the total derivative $\frac{\partial V}{\partial\theta}$ solves the linear system

$$\big(I - \beta\,P_\pi\big)\frac{\partial V}{\partial\theta} \;=\; \sum_a \pi(a \mid s)\,\phi(s,a), \tag{8}$$

with $P_\pi(s,s') = \sum_a \pi(a|s)\,P(s'|s,a)$ the policy-weighted transition matrix.

The BHHH outer optimizer approximates the Hessian with the outer product of scores:

$$B_k \;=\; \frac{1}{N}\sum_{i=1}^{N} \psi_i(\theta_k)\,\psi_i(\theta_k)^\top. \tag{9}$$

This is guaranteed positive definite, avoiding the indefinite Hessian issues that plague Newton methods near saddle points.

Algorithm 1: NFXP-NK (Rust 1987, Iskhakov et al. 2016)

Input: Panel $\mathcal{D} = \{(s_i, a_i)\}_{i=1}^N$, features ϕ, transitions P, discount β, scale σ
Initialize θ_0 from data heuristic `// _estimate_initial_params()`
for $k = 0, 1, 2, \ldots$ **do**
  Inner loop: solve $V = TV$ at θ_k `// _solve_inner()`
  begin `// SA phase: safe global convergence`
    for $j = 1, 2, \ldots$ **do**
      $V \leftarrow TV$
      if $\|V - TV\|_\infty < \varepsilon_{switch}$ **then break**
    end
  end
  begin `// NK phase: quadratic local convergence`
    for $j = 1, 2, \ldots$ **do**
      Solve $(I - \beta \nabla T)\,\delta = TV - V; \quad V \leftarrow V + \delta$
      if $\|\delta\|_\infty < \varepsilon_{inner}$ **then break**
    end
  end
  Compute $Q(s, a) = u(s, a; \theta_k) + \beta \sum_{s'} P(s'|s, a)\, V(s')$
  Compute $\pi(a|s) = \text{softmax}(Q/\sigma)$
  Score: solve $(I - \beta P_\pi)\frac{\partial V}{\partial \theta} = \sum_a \pi \cdot \phi$ `// _compute_analytical_score()`
  Compute ψ_i via (7) for each observation
  Outer update: `// _bhhh_optimize()`
  $B_k \leftarrow \frac{1}{N}\sum_i \psi_i \psi_i^\top + \lambda I$
  $d_k \leftarrow B_k^{-1}\big(\frac{1}{N}\sum_i \psi_i\big)$
  Line search: $\alpha_k \leftarrow$ step-halving until $\ell(\theta_k + \alpha_k d_k) > \ell(\theta_k)$
  $\theta_{k+1} \leftarrow \theta_k + \alpha_k\, d_k$
  if $\|\nabla \ell\| < \varepsilon_{outer}$ **then break**
end
Output: $\hat{\theta}$, V^*, π^*, B^{-1} for robust standard errors

Code annotations reference methods in `src/econirl/estimation/nfxp.py`. The inner loop calls `hybrid_iteration()` from `src/econirl/core/solvers.py`. Default tolerances: $\varepsilon_{\text{switch}} = 10^{-3}$, $\varepsilon_{\text{inner}} = 10^{-12}$, $\varepsilon_{\text{outer}} = 10^{-6}$, $\lambda = 10^{-8}$.

5 Simulation

The companion script `papers/primers/nfxp_demo.py` runs NFXP on the Rust bus engine (90 mileage states, 2 actions, $\beta = 0.95$) with 500 individuals observed for 100 periods. The API call is five lines:

```
from econirl.environments.rust_bus import RustBusEnvironment
from econirl.estimation.nfxp import NFXPEstimator
from econirl.preferences.linear import LinearUtility
from econirl.simulation.synthetic import simulate_panel

env = RustBusEnvironment(operating_cost=0.001,
    replacement_cost=3.0, num_mileage_bins=90,
    discount_factor=0.95)
```

```
utility = LinearUtility.from_environment(env)
panel = simulate_panel(env, n_individuals=500,
    n_periods=100, seed=42)
result = NFXPEstimator(se_method="robust").estimate(
    panel, utility, env.problem_spec,
    env.transition_matrices)
```

Results are auto-generated from the companion script. To update, rerun `nfxp_demo.py` followed by `generate_tables.py`.

Table 1: NFXP-NK on the Rust bus engine (90 states, $\beta = 0.95$, 50,000 obs).

Parameter	True	Estimate	SE
Operating cost (θ_c)	0.001	0.001044	(0.000253)
Replacement cost (RC)	3.0	3.0079	(0.0312)
Log-likelihood			-10156.55
Policy accuracy			100.0%
Time (seconds)			2.66

Both parameters are recovered within one standard error of the true values. The operating cost estimate $\hat{\theta}_c = 0.001044$ is close to the true value of 0.001, and the replacement cost $\widehat{RC} = 3.008$ is close to 3.0. Policy accuracy is 100 percent: the estimated policy assigns the same argmax action as the true policy at every state. Estimation takes under 3 seconds on a single CPU.

Cross-estimator comparison. The Rust bus benchmark runs all twelve estimators on this problem. NFXP, CCP, and MCE-IRL recover identical parameters to four decimal places, confirming DDC-IRL equivalence. Neural estimators (NNES, TD-CCP, SEES) recover parameters with small approximation error. Adversarial estimators (AIRL, GAIL) produce noisier estimates due to optimization variance from the min-max training loop. NFXP's advantages are threefold: the inner loop produces an exact fixed point so the likelihood surface is smooth, the analytical score (8) gives exact gradients at negligible cost, and the BHHH outer product (9) produces valid robust standard errors without bootstrap. The limitation is computational scaling: the inner loop costs $O(|\mathcal{S}|^2)$ per contraction and the NK step costs $O(|\mathcal{S}|^3)$. For state spaces above a few thousand states, NNES, TD-CCP, and SEES become faster because their cost scales with data size rather than state space size.

Counterfactual exercise. The estimated $\hat{\theta}$ supports policy counterfactuals by re-solving (4) at modified parameters. A fuel tax that increases operating costs by 50 percent ($\theta_c' = 1.5\,\hat{\theta}_c$) shifts the replacement threshold to lower mileage, with welfare effect $\Delta W = \mathbb{E}_{\mu_0}[V(s;\theta') - V(s;\hat{\theta})]$. This is valid because $u = \theta^\top \phi$ pins down the reward and $P(s'|s,a)$ does not depend on θ. Counterfactuals that change P itself, such as a preventive maintenance program that slows mileage accumulation, require re-estimating transition probabilities from new data and re-solving the model. The econirl post-estimation module computes ΔW with bootstrap confidence intervals given any pair of parameter vectors $(\hat{\theta}, \theta')$. Standard errors on the counterfactual policy $\pi'(a|s;\theta')$ propagate from the asymptotic distribution of $\hat{\theta}$ via the delta method, since the mapping $\theta \mapsto \pi(\cdot|\cdot;\theta)$ is smooth at any interior fixed point. The companion script `nfxp_demo.py` demonstrates this workflow end to end. Full documentation is at `docs/estimators/nfxp.md`.